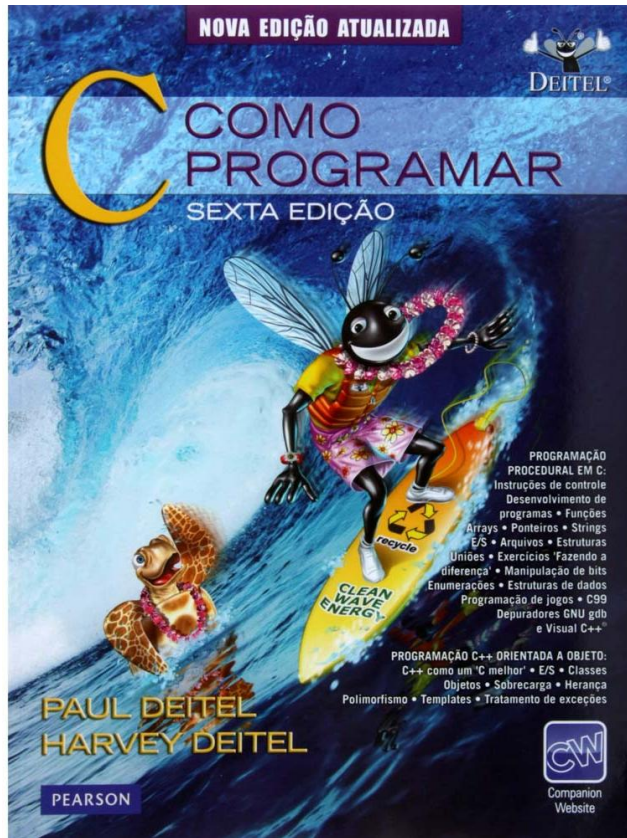


Estruturas de Dados II

- Manipulação de Arquivos (revisão)
- Tipo de dados: Registros (structs) (revisão)



Bibliografia



Título: Como Programa C – Sexta Edição

Editora: Pearson

Autor: Paul Deitel

Capítulo 11.

Manipulação de Arquivo

- O armazenamento de dados em variáveis e arrays é temporário; **todos os dados são perdidos quando um programa termina.**
- Os arquivos são usados para conservação permanente de grandes quantidades de dados.
- Os computadores armazenam arquivos em dispositivos secundários de armazenamento, especialmente dispositivos de armazenamento em disco.

Manipulação de Arquivo

- A linguagem C visualiza cada arquivo simplesmente como um fluxo sequencial de bytes.
- Cada arquivo termina ou com um marcador de final de arquivo (**end-of-file marker**),



Manipulação de Arquivo

- Abrir um arquivo retorna um **ponteiro** para uma estrutura FILE (**definida em <stdio.h>**) que contém informações usadas para processar o arquivo.
- O padrão de entrada, o padrão de saída e o padrão de erros são manipulados por meio dos ponteiros de arquivos **stdin**, **stdout** e **stderr**.

Modos de Abertura

- Para criar um arquivo ou eliminar o conteúdo de um arquivos antes da gravação dos dados, abra o arquivo para gravação ("w").
- Para ler um arquivo existente, abra-o para leitura ("r").
- Para adicionar registros ao final de um arquivo existente, abra o archive append("a").
- Para abrir um arquivo de forma que ele possa ser gravado e lido, abra o arquivo com um dos três modos de atualização "r+", "w+" ou "a+".
 - O modo "r+" abre um arquivo para leitura e gravação.
 - O modo "w+" cria um arquivo para leitura e gravação. Se o arquivo já existir, o arquivo é aberto e o conteúdo atual é eliminado.
 - O modo "a+" abre um arquivo para leitura e gravação. Toda gravação é feita no final do arquivo. Se o arquivo não existir, é criado.

Modos de Abertura (resumo)

"r"

- Abre um arquivo texto para leitura. O arquivo deve existir antes de ser aberto.

"w"

- Abrir um arquivo texto para gravação. Se o arquivo não existir, ele será criado. Se já existir, o conteúdo anterior será destruído.

"a"

- Abrir um arquivo texto para gravação. Os dados serão adicionados no fim do arquivo ("append"), se ele já existir, ou um novo arquivo será criado, no caso de arquivo não existente anteriormente.

"r+"

- Abre um arquivo texto para leitura e gravação. O arquivo deve existir e pode ser modificado.

"w+"

- Cria um arquivo texto para leitura e gravação. Se o arquivo existir, o conteúdo anterior será destruído. Se não existir, será criado.

"a+"

- Abre um arquivo texto para gravação e leitura. Os dados serão adicionados no fim do arquivo se ele já existir, ou um novo arquivo será criado, no caso de arquivo não existente anteriormente.

"...b"

- Executa a mesma operação, só que o arquivo é binário.. Ex. a+b, wb, r+b, etc

fopen

- **Fopen**
- Esta é a função de abertura de arquivos. Seu protótipo é:

```
FILE *fopen (char *nome_do_arquivo, char *modo);
```


fclose

- **fclose**
- Quando acabamos de usar um arquivo que abrimos, devemos fechá-lo. Para tanto usa-se a função **fclose()**:

```
int fclose (FILE *fp) ;
```

feof

- **feof**
- EOF ("End of file") indica o fim de um arquivo.
- Ela retorna não-zero se o arquivo chegou ao EOF, caso contrário retorna zero. Seu protótipo é:

```
int feof (FILE *fp) ;
```

fscan

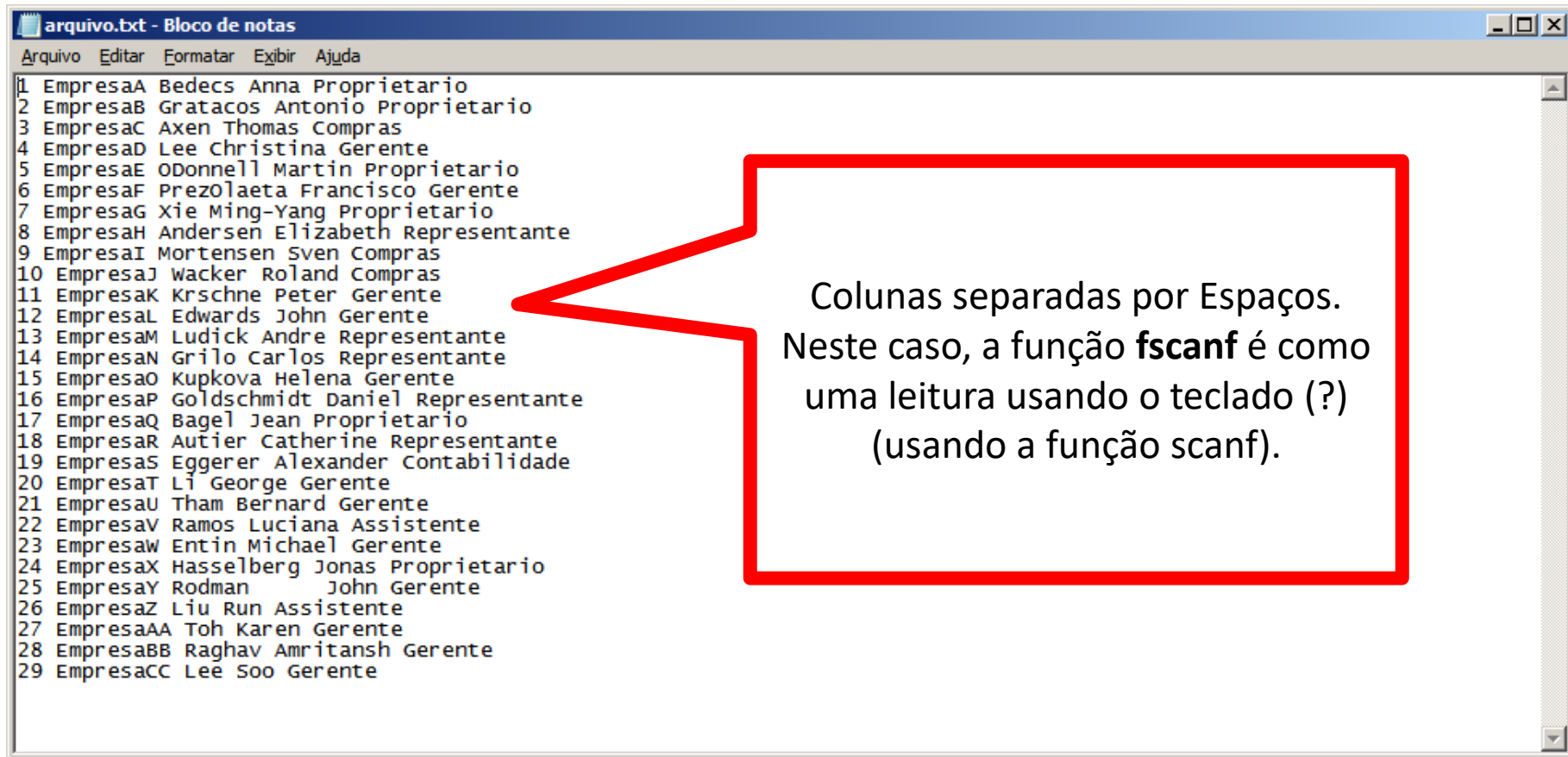
- **fscanf**
- A função **fscanf()** funciona como a função **scanf()**. A diferença é que **fscanf()** lê de um arquivo e não do teclado do computador.
- Protótipo:

```
int fscanf (FILE *fp, char *str, variável) ;
```

EXEMPLO 01 – Lendo Arquivo Exemplo

(Importante, o arquivo exemplo tem 6 colunas)

Layout do Arquivo



```
arquivo.txt - Bloco de notas
Arquivo  Editar  Formatar  Exibir  Ajuda
1 EmpresaA Bedecs Anna Proprietario
2 EmpresaB Gratacos Antonio Proprietario
3 EmpresaC Axen Thomas Compras
4 EmpresaD Lee Christina Gerente
5 EmpresaE ODonnell Martin Proprietario
6 EmpresaF Prezolaeta Francisco Gerente
7 EmpresaG Xie Ming-Yang Proprietario
8 EmpresaH Andersen Elizabeth Representante
9 EmpresaI Mortensen Sven Compras
10 EmpresaJ Wacker Roland Compras
11 EmpresaK Krschne Peter Gerente
12 EmpresaL Edwards John Gerente
13 EmpresaM Ludick Andre Representante
14 EmpresaN Grilo Carlos Representante
15 EmpresaO Kupkova Helena Gerente
16 EmpresaP Goldschmidt Daniel Representante
17 EmpresaQ Bagel Jean Proprietario
18 EmpresaR Autier Catherine Representante
19 EmpresaS Eggerer Alexander Contabilidade
20 EmpresaT Li George Gerente
21 EmpresaU Tham Bernard Gerente
22 EmpresaV Ramos Luciana Assistente
23 EmpresaW Entin Michael Gerente
24 EmpresaX Hasselberg Jonas Proprietario
25 EmpresaY Rodman John Gerente
26 EmpresaZ Liu Run Assistente
27 EmpresaAA Toh Karen Gerente
28 EmpresaBB Raghav Amritansh Gerente
29 EmpresaCC Lee Soo Gerente
```

Colunas separadas por Espaços.
Neste caso, a função **fscanf** é como
uma leitura usando o teclado (?)
(usando a função scanf).

Lendo Arquivo Sequencial (fscanf)

```
#include <stdio.h>
#include <stdlib.h>

main(){
    int codigo;
    char empresa[10];
    char sobre[12];
    char nome[10];
    char funcao[10];

    FILE *txt;

    if((txt = fopen("arquivo.colunas","r")) == NULL)
        {
            printf("Erro ao abrir arquivo");
        }
    else
    {
        while (!feof(txt)) {
            fscanf(txt, "%d %s %s %s %s ", &codigo, empresa, sobre, nome, funcao);
            printf("%d \t %-10s %-12s %-10s %-10s \n", codigo, empresa, sobre, nome, funcao);
        }
        fclose(txt);
    }
    system("pause");
}
```

Escrevendo Formatado: fprintf

- **fprintf**
 - A função **fprintf()** funciona como a função **printf()**. A diferença é que a saída de **fprintf()** é um arquivo e não a tela do computador.
- Protótipo:

```
int fprintf (FILE *fp, char *str, ...);
```

EXEMPLO 02 – Escrevendo em um Arquivo (a+)

EXEMPLO 03 – Escrevendo em um Arquivo (w)

Leitura Byte a Byte: fgetc

- **Fgetc**

- Lê o caractere presente na posição atual do fluxo interno. Após a leitura, a posição atual é avançada para o próximo caractere.

- Protótipo:

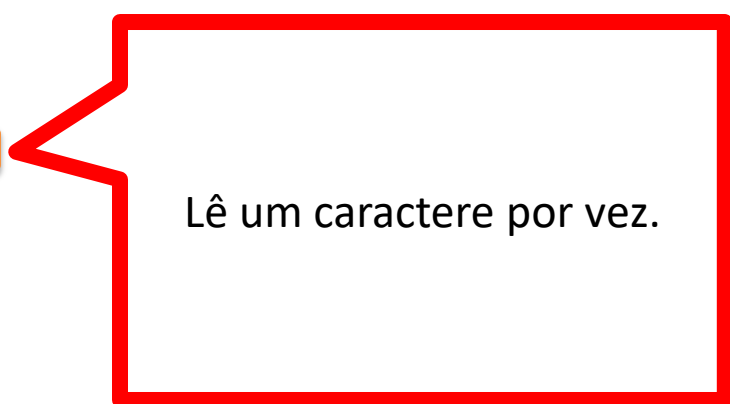
```
int fgetc (FILE * fluxo) ;
```

EXEMPLO 04 – Lendo um Arquivo **(caractere a caractere)**

Lendo Arquivo Sequencial Não Formatado

```
#include <stdio.h>
#include <stdlib.h>

main()
{
    char caractere;
    FILE *txt;
    if((txt = fopen("c:\\arquivo.txt", "r")) == NULL)
    {
        printf("Erro ao abrir arquivo");
    }
    else
    {
        while (!feof(txt)) {
            caractere = fgetc(txt);
            printf("%c", caractere);
        }
        fclose(txt);
    }
    system("pause");
}
```



Lê um caractere por vez.

Inserindo Barra “|”entre as palavras.

```
#include <stdio.h>
#include <stdlib.h>

main()
{
    char caractere;

    FILE *txt;

    if((txt = fopen("c:\\arquivo.txt","r")) == NULL)
    {
        printf("Erro ao abrir arquivo");
    }
    else
    {
        while (!feof(txt)) {
            caractere = fgetc(txt);
            printf("%c", caractere);
        }
        fclose(txt);
    }
    system("pause");
}
```

Lê um caractere por vez.

```
if (caractere == ' ')
    printf(" | ");
else
    printf("%c", caractere);
```

Lendo conjunto de bytes: fgets

- **fgets**

- Lê do *fluxo* para a cadeia de caracteres *string* até a quantidade de caracteres (*tamanho* - 1) ser lida ou até uma nova linha (`\n`) ou EOF ser encontrado. Após a leitura, a posição atual do *fluxo* é avançada para o próximo caractere não lido.
 - Para ler da entrada padrão (stdin) de forma segura, é necessário o uso desta função. Como ela limita o número de caracteres lidos pelo parâmetro *tamanho*, ela previne buffer overflows que possam causar erros de segurança ou *crashes* na aplicação.
- A função fgets **pára** caso encontre uma nova linha (`\n`), incluindo-a na *string*.
 - A função lê até o (*tamanho* - 1) pois devemos contar o espaço para o NULL (`'\0'`) no final da *string*.

- **Protótipo:**

```
char * fgets (char * string, int tamanho, FILE * fluxo);
```

EXEMPLO 05 – Lendo um arquivo (conjunto de de caracteres ou parágrafo)

Lendo Arquivo Sequencial (fgets)

```
#include <stdio.h>
#include <stdlib.h>

main()
{
    char linha[1024];

    FILE *txt;

    if((txt = fopen("arquivo_texto.txt","r")) == NULL)
    {
        printf("Erro ao abrir arquivo");
    }
    else
    {
        while (!feof(txt)) {
            fgets(linha, 1024, txt);
            printf("%s", linha);
        }
        fclose(txt);
    }
    system("pause");
}
```

Lê uma string até o **\n** ou **1023** caracteres;

Lendo Arquivo Sequencial (fgets)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
main() {
    char linha[1024];
    char * ultima;
    FILE *txt;
    if((txt = fopen("c:\\arquivo.txt","r")) == NULL)
    {
        printf("Erro ao abrir arquivo");
    }
    else {
        while (!feof(txt)) {
            fgets(linha, 1024, txt);
            ultima = strtok (linha, " ");
            while (ultima != NULL)
            {
                printf ("%s ",ultima);
                ultima = strtok (NULL, " ");
            }
            fclose(txt);}
        system("pause"); }
```

strtok quebra a string no delimitador. (neste caso, separa em palavras)

Tipo de dados:

Registros (structs)

```
typedef struct point
{
    int x;
    int y;
} Point;
```

Typedef in C language



TIPOS DE DADOS

TIPOS DE DADOS PRIMITIVOS

A cada variável está associado um Tipo de Dados. O tipo de dado define quais os valores que a variável pode conter. Se, por exemplo, dissermos que uma variável é do tipo Inteiro, não poderemos lá colocar um valor Real ou um Caractere.

Ao declararmos o tipo de dados de uma variável, estamos a definir, não só, o tipo de valores que esta pode conter, mas também quais as operações que com elas podemos realizar.

Exemplo de tipos primitivos:

- INT
- REAL
- CHAR
- BOOL

TIPOS DE DADOS

TIPOS DE DADOS COMPOSTOS HOMOGÊNEOS

O nome mais comum atribuído as variáveis compostas homogêneas é **vetor ou matrizes**.

Vetores, matrizes ou *arrays* correspondem a um tipo de dado utilizado para representar (armazenar e manipular) **uma coleção de valores do mesmo tipo**.

Uma outra definição para vetores ou matrizes corresponde a um conjunto de variáveis, do mesmo tipo, identificadas por um único nome, onde cada variável (posição) é referenciada por meio de número chamado de índice.

Os colchetes são utilizados para conter o índice.

TIPOS DE DADOS

TIPOS DE DADOS COMPOSTOS HOMOGÊNEOS

As variáveis compostas homogêneas são estruturas de dados que caracterizam-se por um conjunto de variáveis do mesmo tipo.

Elas pode ser unidimensionais ou multidimensionais.

Unidimensionais (vetores):

A variável composta homogênea unidimensional caracteriza-se por dados agrupados linearmente numa única direção, como uma linha reta.

Multidimensionais (matrizes):

A variável composta multidimensional caracteriza-se por dados agrupados em diferentes direções, como em um cubo;

TIPOS DE DADOS

TIPOS DE DADOS COMPOSTOS HETEROGÊNEOS:

As estruturas **heterogêneas** são conjuntos de dados formados por tipos de **dados primitivos diferentes** (campos do registro) em uma mesma estrutura.

REGISTROS - STRUCT

As estruturas de dados consistem em criar apenas um dado que contém vários membros, que nada mais são do que outras variáveis.

De uma forma mais simples, é como se uma variável tivesse outras variáveis dentro dela.

A vantagem em se usar estruturas de dados é que podemos agrupar de forma organizada vários tipos de dados diferentes, por exemplo, dentro de uma estrutura de dados podemos ter juntos tanto um tipo float, um inteiro, um char ou um double.

As variáveis que ficam dentro da estrutura de dados são chamadas de membros.

```
struct nome
{
    •      tipo1 variavel1;
    •      tipo2 variavel2;
        tipo3 variavel3;
        . . .
};
```

REGISTROS

Exemplo de Criação:

```
#include <stdio.h>
#include <string.h>
```

```
typedef struct Cpessoa
{
    char nome[20];
    int idade;
};
```

Definição do Registro (struct)

```
int main(void)
{
```

```
    Cpessoa aluno[4];
```

<- Definição do Vetores de Registro (4 posições)

```
    strcpy(aluno[0].nome, "Celso de Melo");
    aluno[0].idade = 22;
```

```
    strcpy(aluno[1].nome, "Augusto Nunes");
    aluno[1].idade = 22;
```

```
    strcpy(aluno[2].nome, "Julia Cleria");
    aluno[2].idade = 23;
```

```
    strcpy(aluno[3].nome, "Lucas Barroso");
    aluno[3].idade = 24;
```

Manipulação

```
    for (int i = 0; i<4 ; i++)
    {
```

```
        printf("%s \t %d \n", aluno[i].nome, aluno[i].idade);
```

```
    }
    getchar();
```

Impressão

```
}
```

REGISTROS

Ponteiro de Struct

Um struct consiste em vários dados agrupados em apenas um. Para acessarmos cada um desses dados, usamos um ponto (.) para indicar que o nome seguinte é o nome do membro.

Exemplo:

```
aluno[1].idade
```

Um ponteiro guarda o endereço de memória que pode ser acessado diretamente.

```
paluno->idade
```


Registros

```
...  
typedef struct Cpessoa  
{  
    char nome[20];  
    int idade;  
};
```

Criação do Registro (struct)

```
int main(void)  
{
```

```
    Cpessoa aluno[4];
```

```
    Cpessoa *paluno = aluno;
```

Criação do ponteiro para a struct

```
    strcpy(paluno->nome, "Celso de Melo");
```

```
    paluno->idade = 22;
```

```
    paluno++;
```

```
    strcpy(paluno->nome, "Augusto Nunes");
```

```
    paluno->idade = 22;
```

```
    paluno++;
```

Avança uma posição de memória

```
    strcpy(paluno->nome, "Julia Cleria");
```

```
    paluno->idade = 23;
```

```
    paluno++;
```

```
    strcpy(paluno->nome, "Lucas Barroso");
```

```
    paluno->idade = 24;
```

```
    paluno++;
```

Volta para a Posição inicial do
ponteiro

```
    paluno = aluno;
```

```
    for (int i = 0; i < 4; i++)
```

```
    {
```

```
        printf("%s \t %d \n", paluno->nome, paluno->idade);
```

```
        paluno++;
```

```
    }
```

```
    getchar();
```

```
}
```

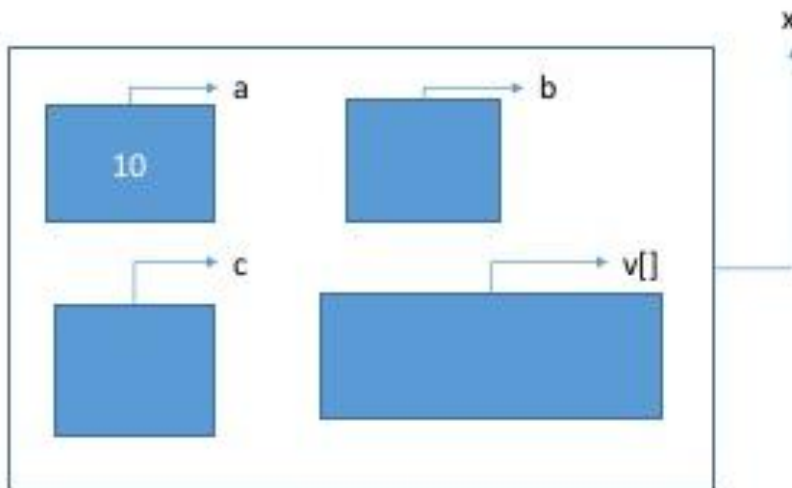
Uma possível representação visual para uma struct seria:

```
struct {  
    int a;  
    char b;  
    float c;  
    int v[5];  
} x;
```

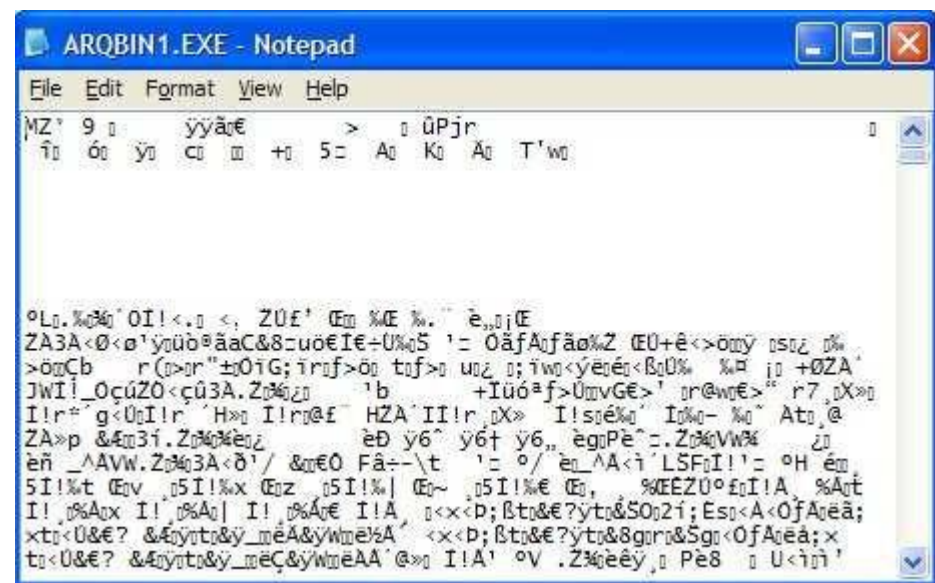
Se fizermos:

`x.a = 10;`

A caixinha de a
receberá o
valor 10!



Arquivos Binários



EXEMPLO 06 – Escrevendo Arquivos Binários.

fwrite

FWRITE

- fwrite tenta escrever para o *fluxo* *numero_itens* elementos, com *tamanho* bytes cada.
- Em caso de sucesso, ou seja, todos os elementos tenham sido escritos com sucesso, fwrite escreveu (*tamanho* * *numero_itens*) bytes do parâmetro *dados* para o *fluxo*.
- fwrite funciona como se *fputc* fosse chamada *tamanho* vezes para cada objeto.
- O ponteiro interno de posição do *fluxo* é avançado pelo número de bytes escritos com sucesso.
- A marca temporal de última modificação do arquivo é atualizada.

Protótipo:

```
size_t fwrite(void * dados, size_t tamanho, size_t numero_itens, FILE * fluxo);
```

```
#include <stdio.h>
#include <string.h>
typedef struct Cpessoa
{   char nome[20];
    int idade; };

int main(void)
{
    char condicao = 's';
    Cpessoa aluno;
    FILE *bin;

    if((bin = fopen("arquivo_binario.txt","ab")) == NULL)
    {
        printf("Erro ao abrir arquivo");    }
    else
    {
        while (condicao == 's' || condicao == 'S')
        {
            printf("Informe o nome:");
            scanf("%s", aluno.nome);

            printf("Informe a idade:");
            scanf("%d", &aluno.idade);

            printf("Continuar S/N?:");

            fwrite(&aluno, sizeof(aluno), 1, bin);

            fflush(stdin);
            condicao = getchar();
        }
    }
}
```

EXEMPLO 07 – Lendo Arquivos Binários.

fread

FREAD

- fread tenta ler do *fluxo* *numero_itens* elementos, com *tamanho* bytes cada. Em caso de sucesso, ou seja, todos os elementos tenham sido lidos com sucesso, fread lê (*tamanho* * *numero_itens*) bytes do *fluxo* para o parâmetro *dados*.
- fread funciona como se [fgetc](#) fosse chamada *tamanho* vezes para cada objeto. Note que fread apenas *funciona como* chamadas sucessivas a fgetc. Na realidade, fread não faz uso da função fgetc.
- O ponteiro interno de posição do *fluxo* é avançado pelo número de bytes lidos.

Protótipo:

```
size_t fread(void * dados, size_t tamanho, size_t numero_itens,  
             FILE * fluxo);
```



```
#include <stdio.h>
#include <string.h>

typedef struct Cessoa
{
    char nome[20];
    int idade;};

int main(void)
{
    char condicao = 's';
    Cessoa aluno[200];
    FILE *bin;
    int tamanho = 0;

    if((bin = fopen("arquivo_binario.txt","rb")) == NULL)
    {
        printf("Erro ao abrir arquivo");
    }
    else
    {
        //Lendo o Arquivo
        while (!feof(bin))
        {
            fread(&aluno[tamanho] ,sizeof(Cessoa),1, bin);
            tamanho++;
        }
        //Mostrando na tela
        for (int i = 0; i<tamanho-1 ; i++)
        {
            printf("%s \t %d \n", aluno[i].nome, aluno[i].idade);
        }
        getchar();
    }
}
```